

Разработка мобильных приложений 2 семестр

Автор: Леонтьев Д.А.

denis.leontev92@yandex.ru

Проблемы разработки

В одном проекте одновременно участвуют:

- **Android Studio**
- **Gradle**
- **Android Gradle Plugin (AGP)**
- **Kotlin**
- **JDK**
- **SDK / build-tools / platform-tools**
- **плагины (KMP, Compose, Hilt и т.д.)**

Все они должны быть совместимы между собой, и Google часто:

- ✓ **выпускает новое → старое ломается**
- ✓ **меняет требования без нормальной миграции**

Как будет проходить дисциплина

- **Лекция:** одна вводная
- Основной упор — на **практику**: лабораторные работы в **IntelliJ IDEA** и **Android Studio**.
- Вы изучите:
 - продвинутый **Kotlin**
 - **Android-приложения**
 - возможно в 3 модуле **Flutter**

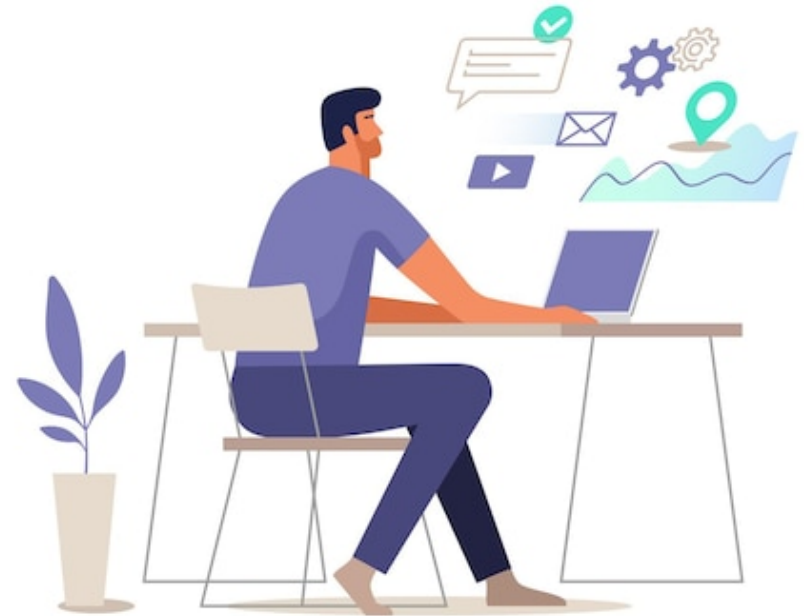
Баллы

- Посещение занятий;
- Работа на парах;
- Выполнение домашних работ;
- Защита проекта.

Мобильные приложения: особенности и типы

Мобильные приложения: чем они отличаются и какими бывают

Не просто «программа в кармане» — а особая среда, особые правила



Контекст использования

Пользователь на ходу: короткие сессии, высокая фрустрация

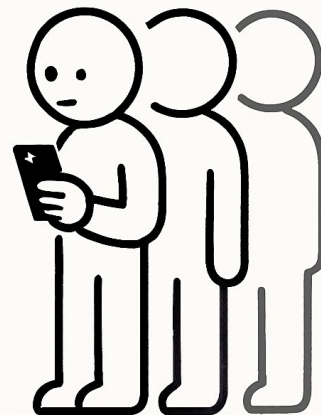
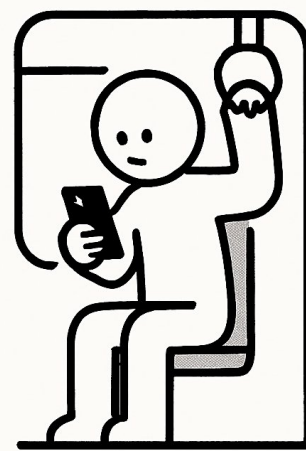
- Короткие сессии (часто < 30 сек)
- Использование «между делом»
- Частые прерывания (звонки, уведомления)
- Ограниченное внимание



В ТРАНСПОРТЕ

НА УЛИЦЕ

В ОЧЕРЕДИ



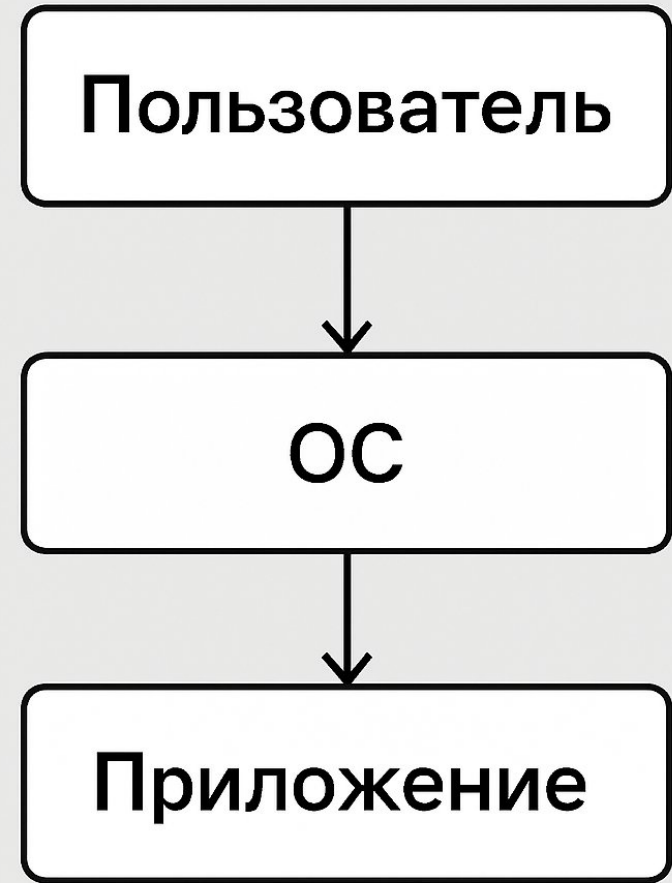
Мобильное vs Десктопное

Десктоп	Мобильное
Долгая сессия	Короткая сессия
Большой экран	Маленький экран
Мышь + клавиатура	Один палец (часто одной рукой)
Стабильные ресурсы	Батарея, память, сеть — ограничены
Пользователь «сидит»	Пользователь «бежит»

Кто на самом деле «хозяин» телефона?

Система управляет всем

- Приложение не контролирует своё выполнение
- ОС может приостановить или убить приложение в любой момент
- Приоритет ОС — батарея, память, безопасность



Три главных ограничения

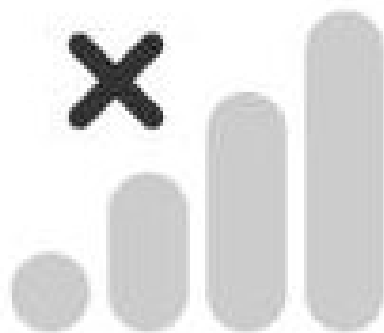
Основные ограничения:



Сеть — ненадёжный друг

Интернет на телефоне — как погода

- Может пропасть внезапно
- Может быть медленным (2G в метро!)
- Может переключаться между Wi-Fi и мобильной сетью



Вывод: что значит «думать как мобильный разработчик»?

Ключевые принципы

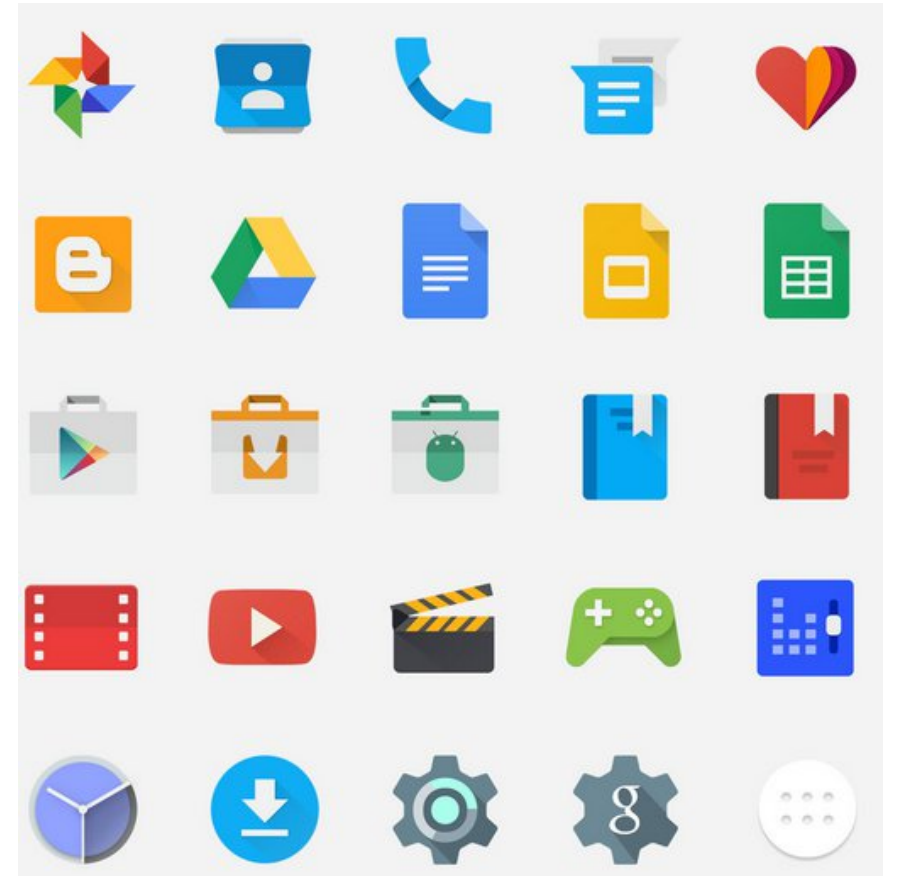
- Быстро решать задачу
- Готовность к прерываниям
- Работа в условиях ограничений
- Пользователь в центре — не функционал

Типы мобильных приложений

Почему важно знать типы приложений

Не все приложения одинаковы

Знание типа = понимание архитектуры, сложности и рисков

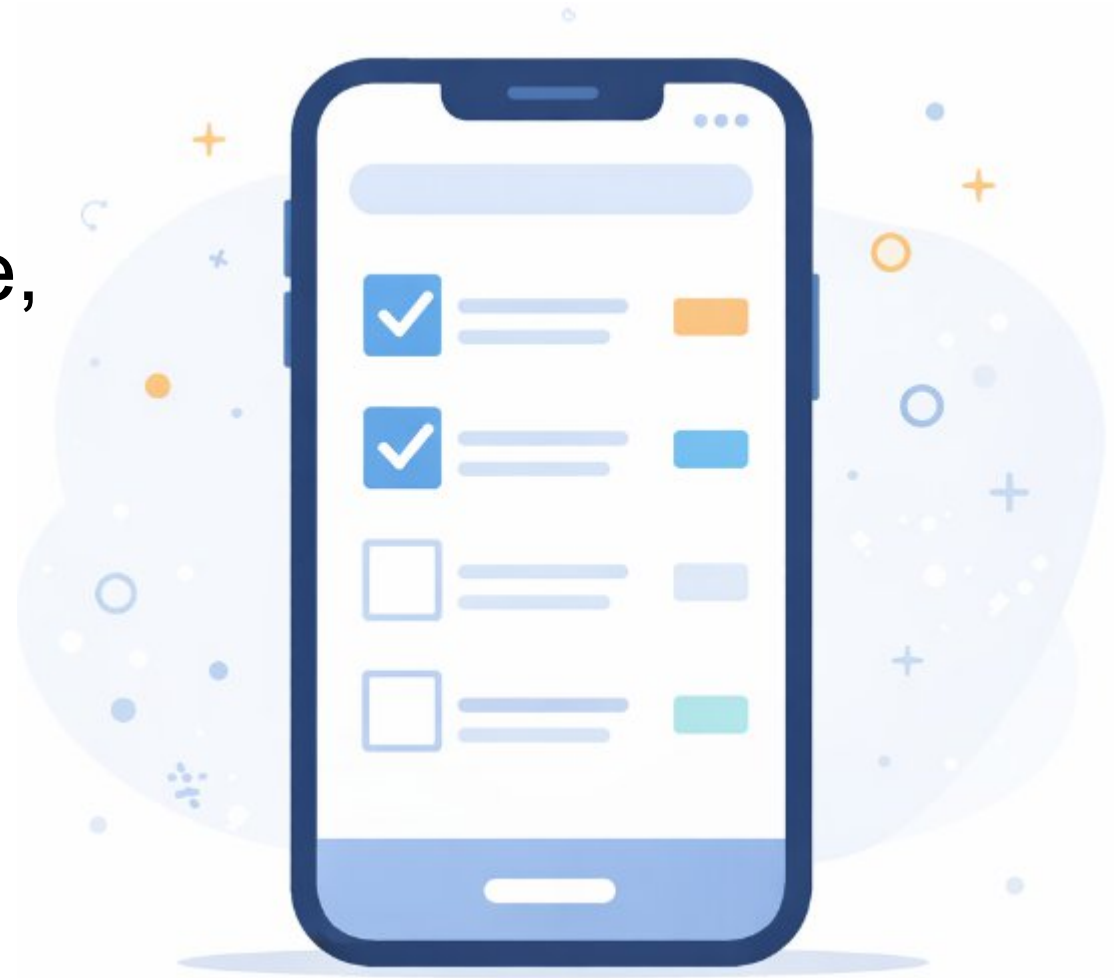


Тип 1: CRUD-приложения

Просто, но фундаментально

- Create, read, update and delete (Создание, чтение, обновление, удаление)
- Списки, заметки, напоминания
- Низкая сложность, локальное хранилище

Примеры: Notes, Todoist (базовый режим), Keep



Тип 2: Клиент–серверные приложения

Большинство реальных приложений

- Данные — на сервере
- Приложение — «окно» в облако
- Проблемы: сеть, синхронизация, ошибки

Примеры: Wildberries, СберБанк, Ozon



Тип 3: Реалтайм-приложения

Где каждая миллисекунда на счету

- Данные обновляются мгновенно
- Требуется постоянное соединение
- Ограничения ОС в фоне — главный враг

Примеры: Telegram, Яндекс.Такси, Uber



Тип 4: Медиа-приложения

Тяжеловесы с высокими требованиями

- Аудио, Видео, Изображения
- Большой объём данных
- Требуют фоновой работы

Примеры: Spotify, YouTube, TikTok

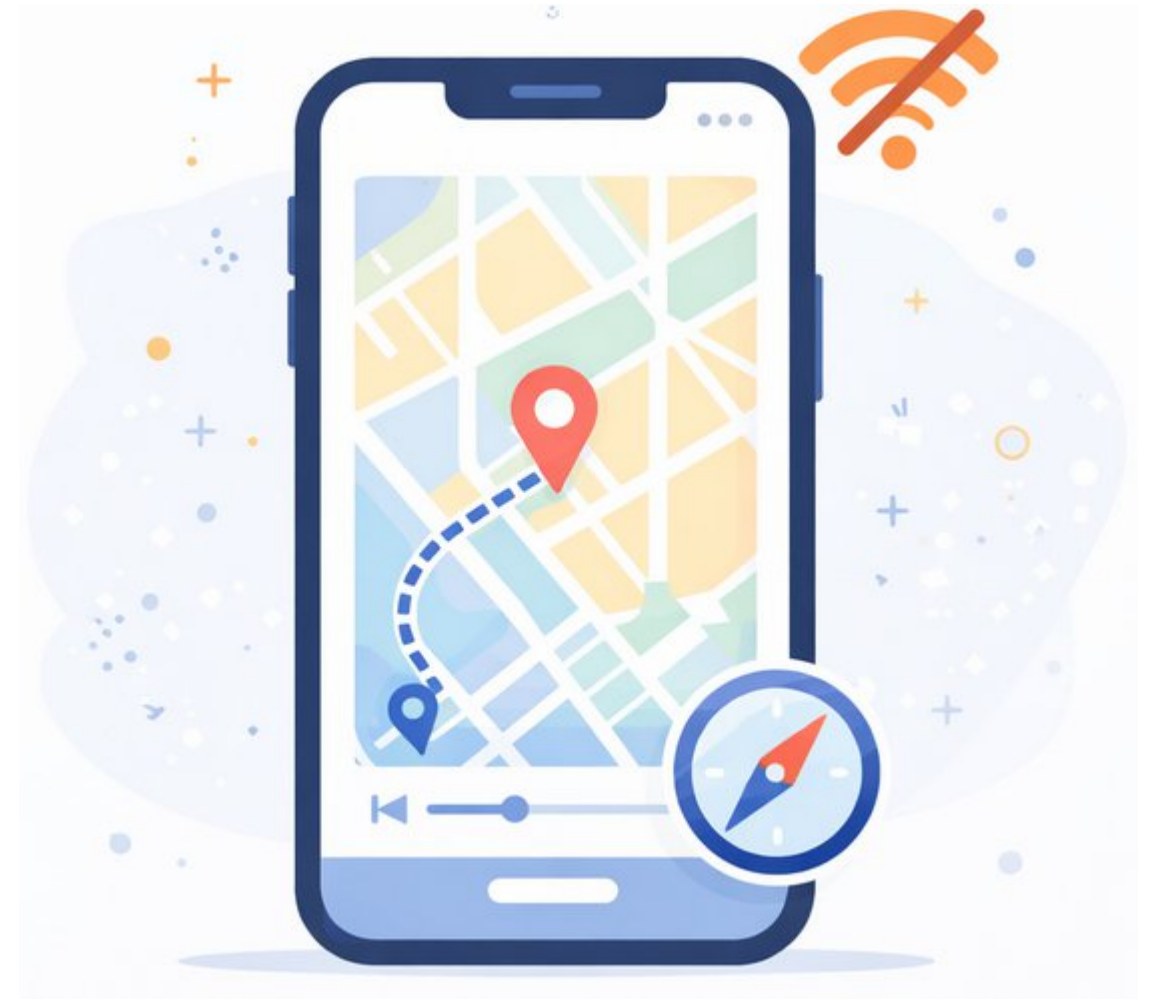


Тип 5: Offline-first приложения

Работают даже в тайге

- Полная функциональность без интернета
- Данные синхронизируются позже
- Конфликты: «кто прав — телефон или сервер?»

Примеры: Google Maps (офлайн-карты), Notion, Obsidian

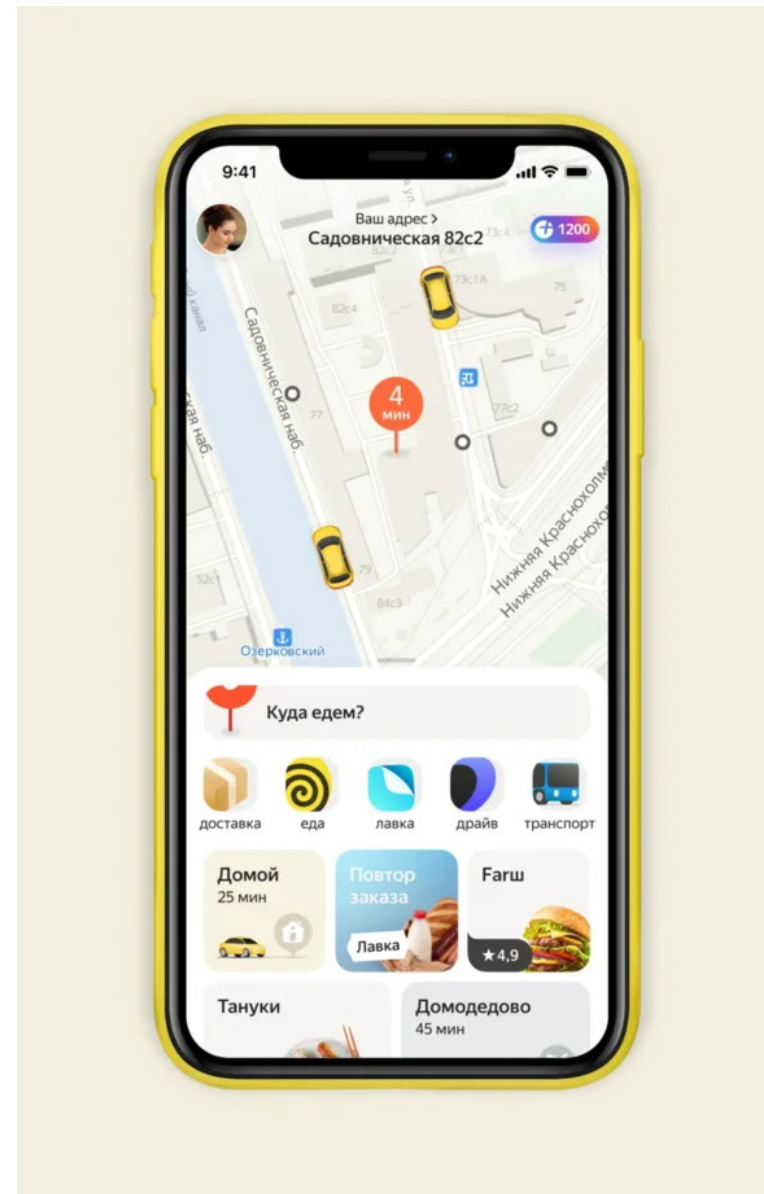


Приложения — гибрид

Реальные проекты — смесь
типов

Такси-приложение =

- клиент-сервер (заказ),
- реалтайм (отслеживание),
- медиа (фото машины),
- offline-first (сохранение истории поездок)



Почему это важно?

От типа зависит всё

- Архитектура (MVC, MVVM, Clean Architecture?)
- Технологический стек
- Безопасность и кеширование
- Сроки разработки и тестирования

Итог: 3 главных мысли

1. Мобильная среда — ограниченная, нестабильная, управляемая ОС
2. Приложение должно работать под пользователя, а не под себя
3. Тип приложения — ключ к архитектуре, сложности и успеху

Вопросы для обсуждения

Попробуйте сами!

К какому типу относится **Telegram**?

реалтайм + медиа + клиент-сервер

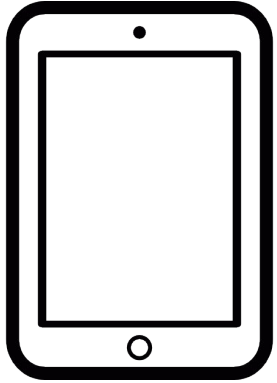
Почему **банковское приложение** сложнее заметок?

безопасность, сеть, валидация

Почему **мобильное приложение** ≠ **мобильный сайт**?

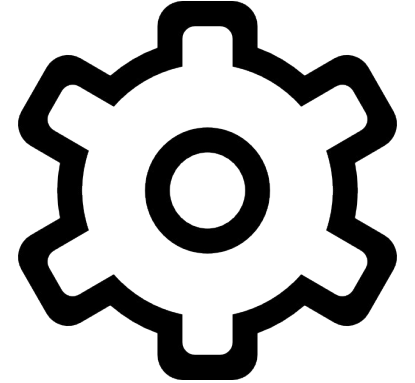
контекст, ресурсы, функционал устройства

Основные компоненты Android



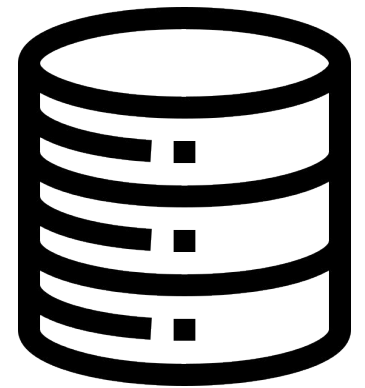
Activities

Services

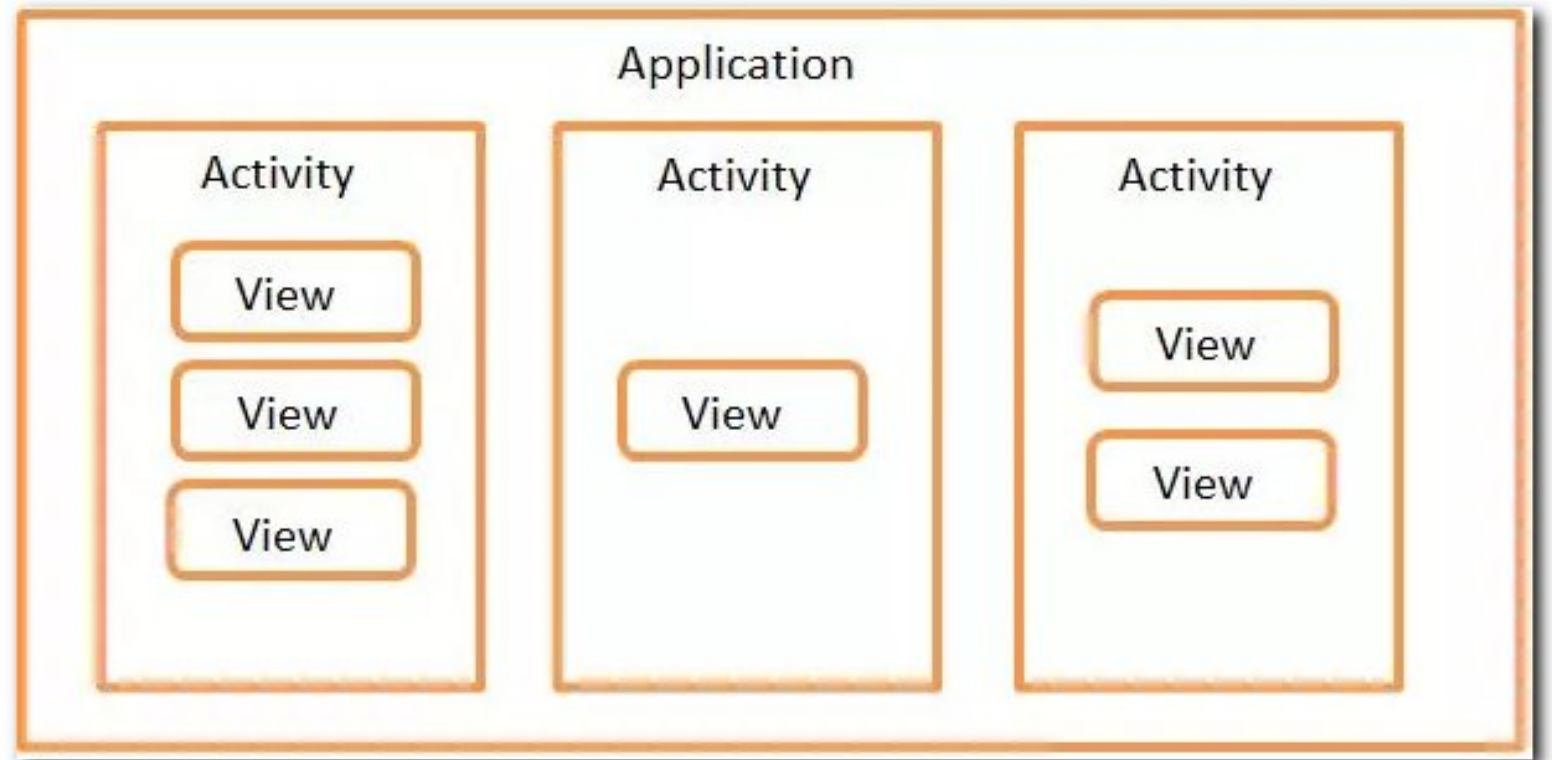
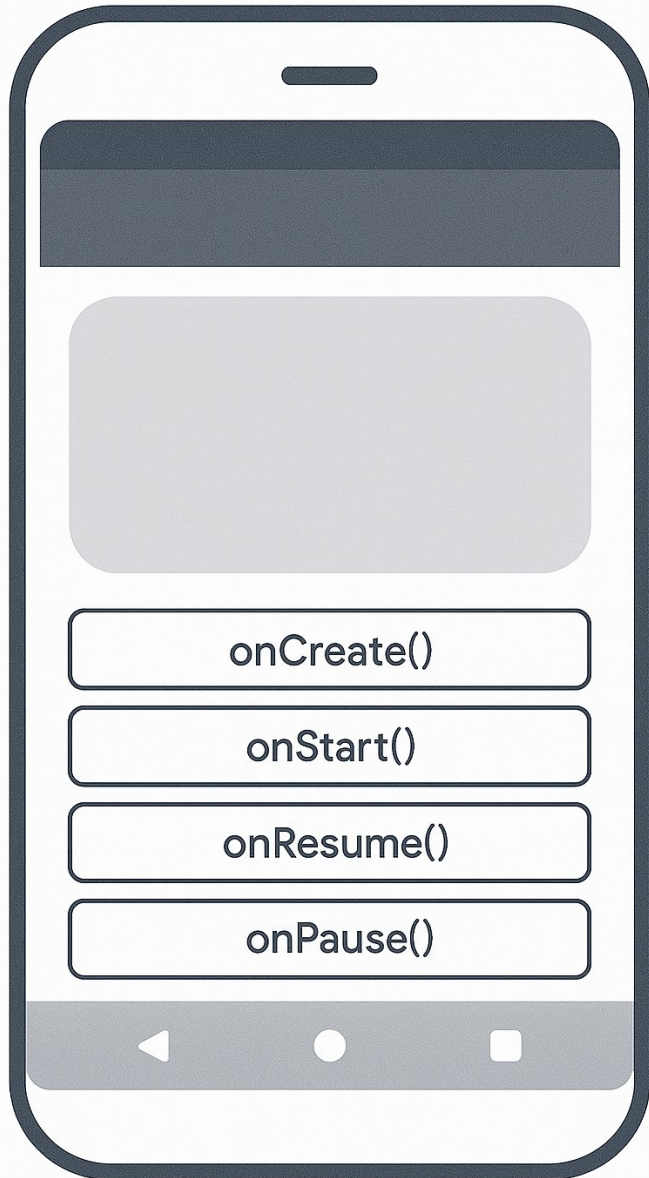


**Broadcast
Receivers**

**Content
Providers**



Activities & the Activity Lifecycle

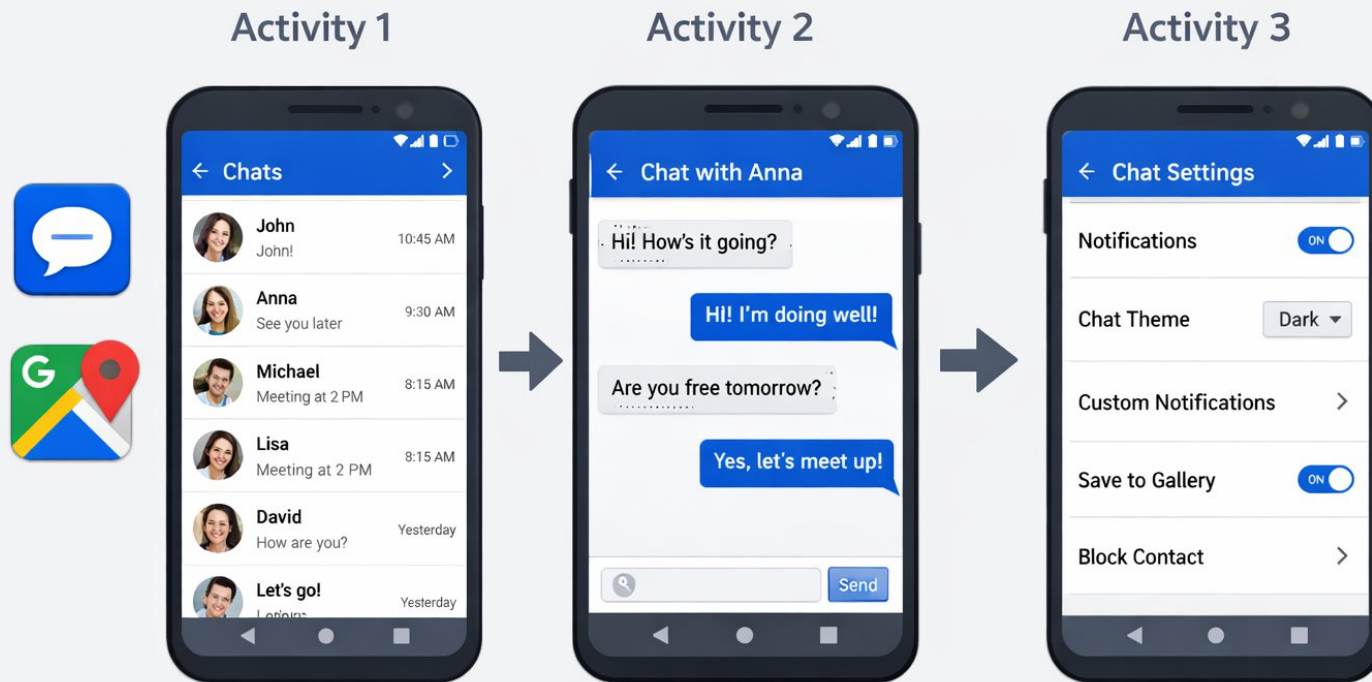


```
class MainActivity : ComponentActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        enableEdgeToEdge()  
        setContent {  
            MyApplicationTheme {  
                Surface(  
                    modifier = Modifier.fillMaxSize()  
                ) {  
                    TipTimeLayout()  
                }  
            }  
        }  
    }  
}
```

Пример Activity в Android Studio

Что такое Activity?

Activity — это экран приложения



Одно Activity = один экран с пользовательским интерфейсом.

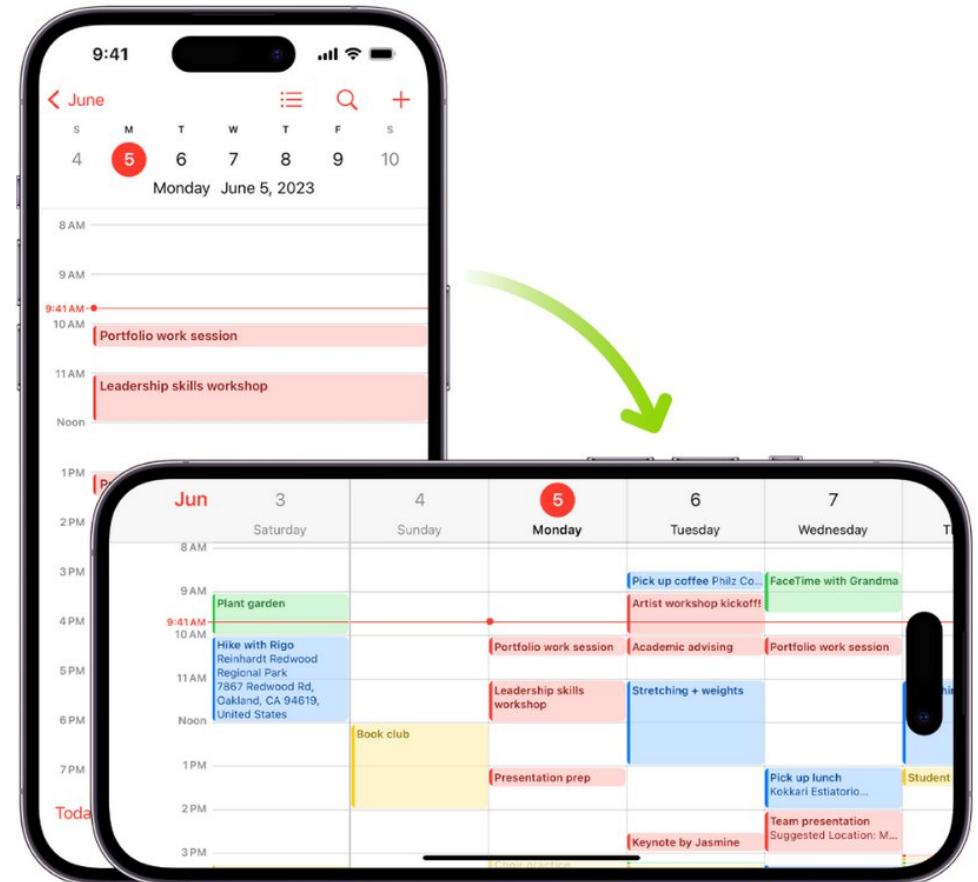
Зачем нужен жизненный цикл?

Мир Android — хаотичен. Пользователь — король

Звонок поверх вашего приложения



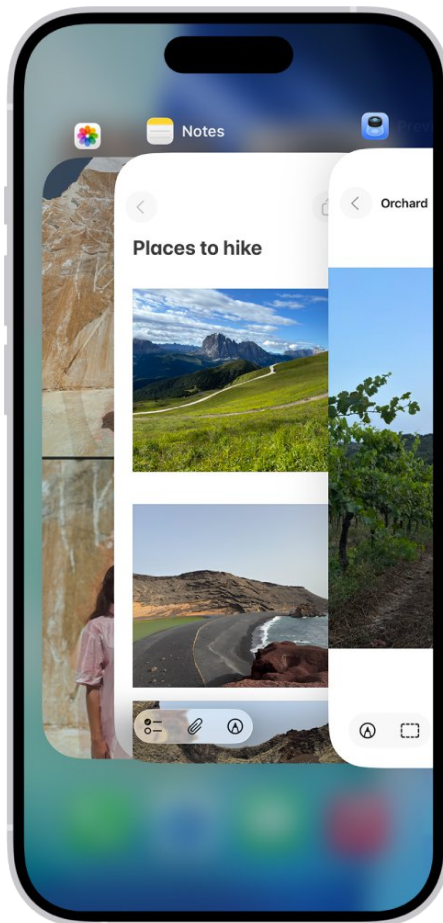
Поворот экрана



Кнопки «Назад»

Back

Сворачивание и
переход в **другое**
приложение



Системе не хватает памяти, и
она начинает «убивать»
фоновые приложения



Вопрос: Как приложение должно адекватно реагировать на это всё?

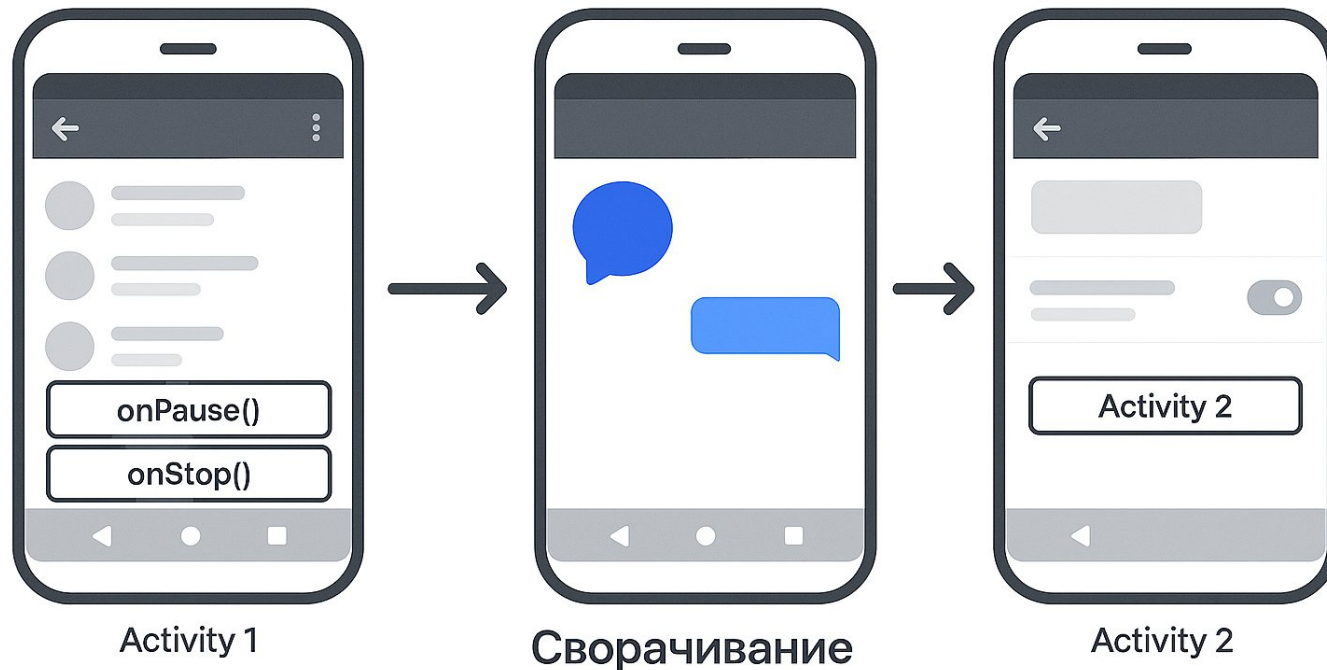
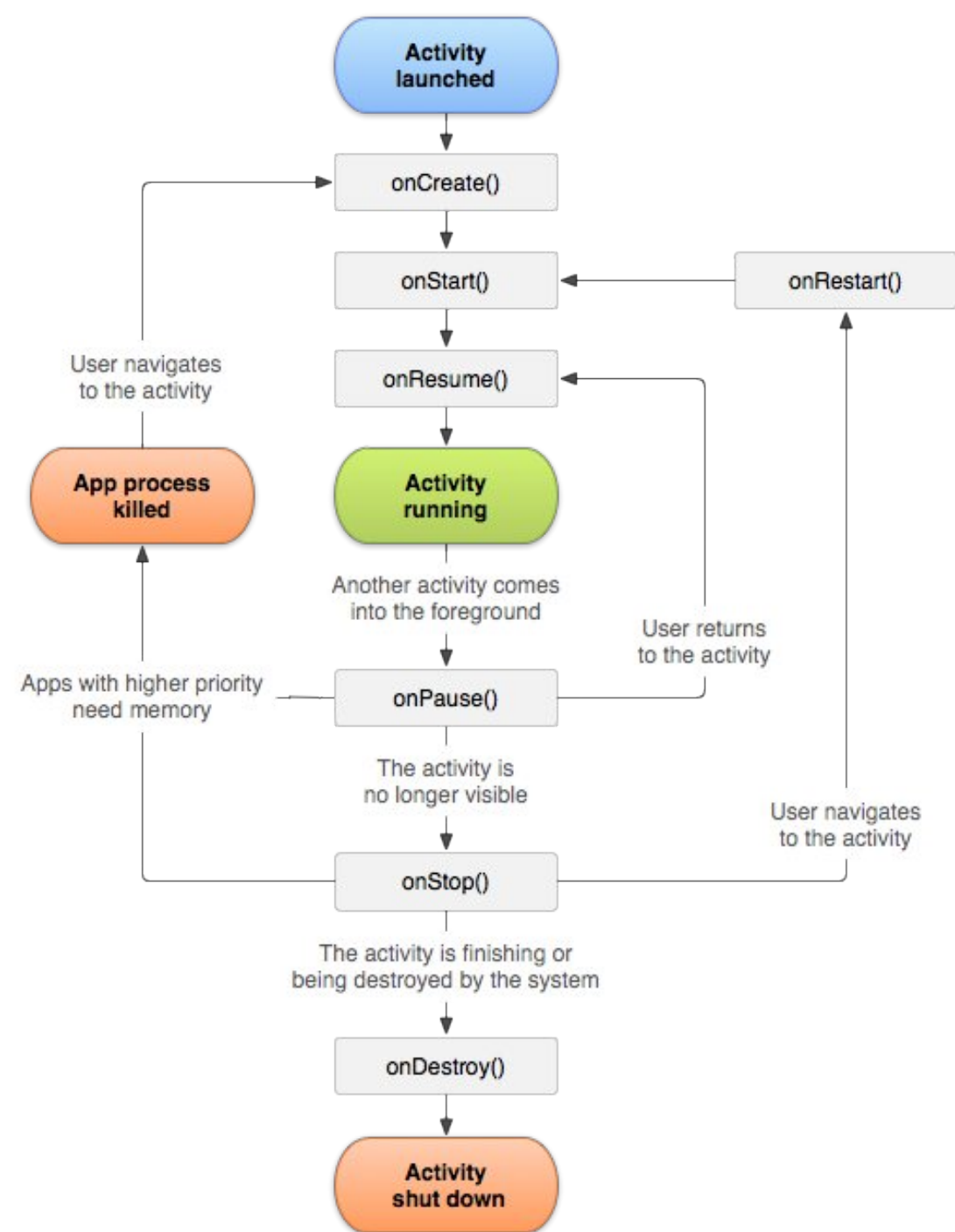
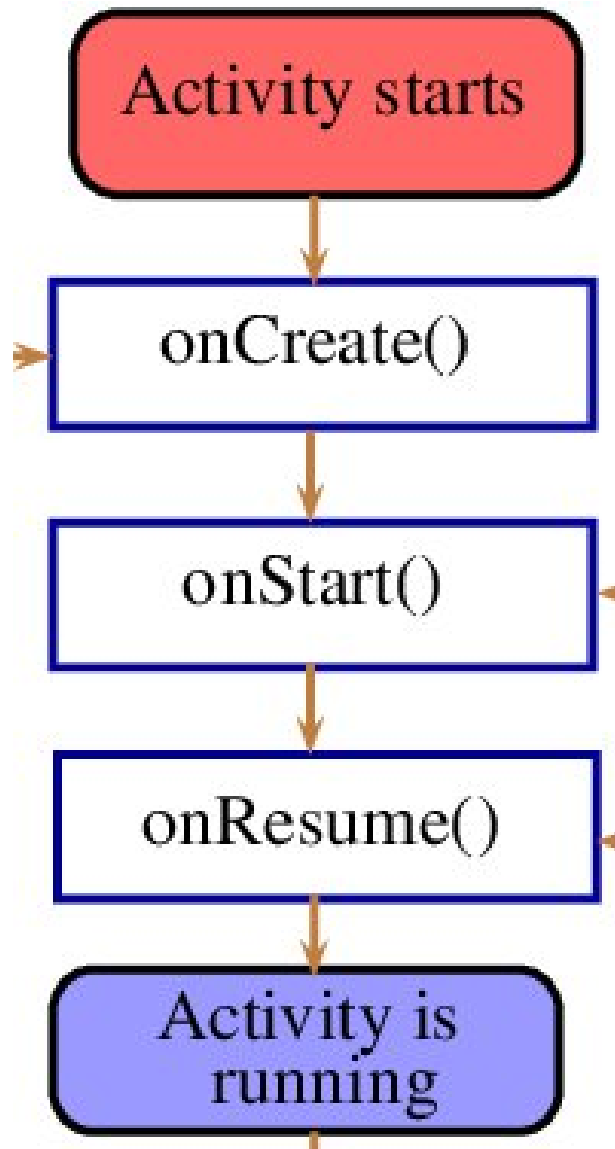


Диаграмма жизненного цикла



Основные методы и «путь создания»

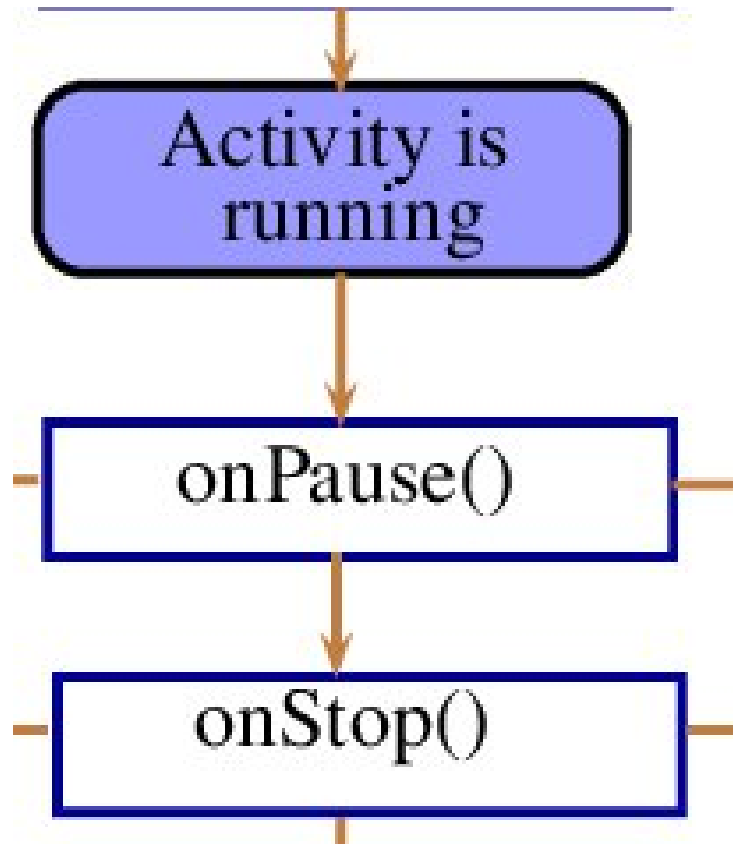


onCreate(): Создание. Вызывается один раз за всю жизнь. Здесь setContentView() (надуваем интерфейс из XML), находим View, восстанавливаем сохраненные данные.

onStart(): «Видимость». Активность становится видимой пользователю, но еще не на переднем плане. Редко переопределяется.

onResume(): «Фокус». Активность на переднем плане, пользователь с ней взаимодействует. Запускаем анимации, камеру, таймеры.

Пауза и остановка («путь ухода в фон»)



onPause(): «Частично перекрыли». Поверх Activity появилось полупрозрачное окно (диалог, звонок). Активность видна, но не в фокусе. Быстрый метод! Здесь сохраняем самые критичные данные.

onStop(): «Полностью скрыли». Активность больше не видна (пользователь перешел в другую или свернул приложение). Здесь освобождаем тяжелые ресурсы (сеть, CPU).

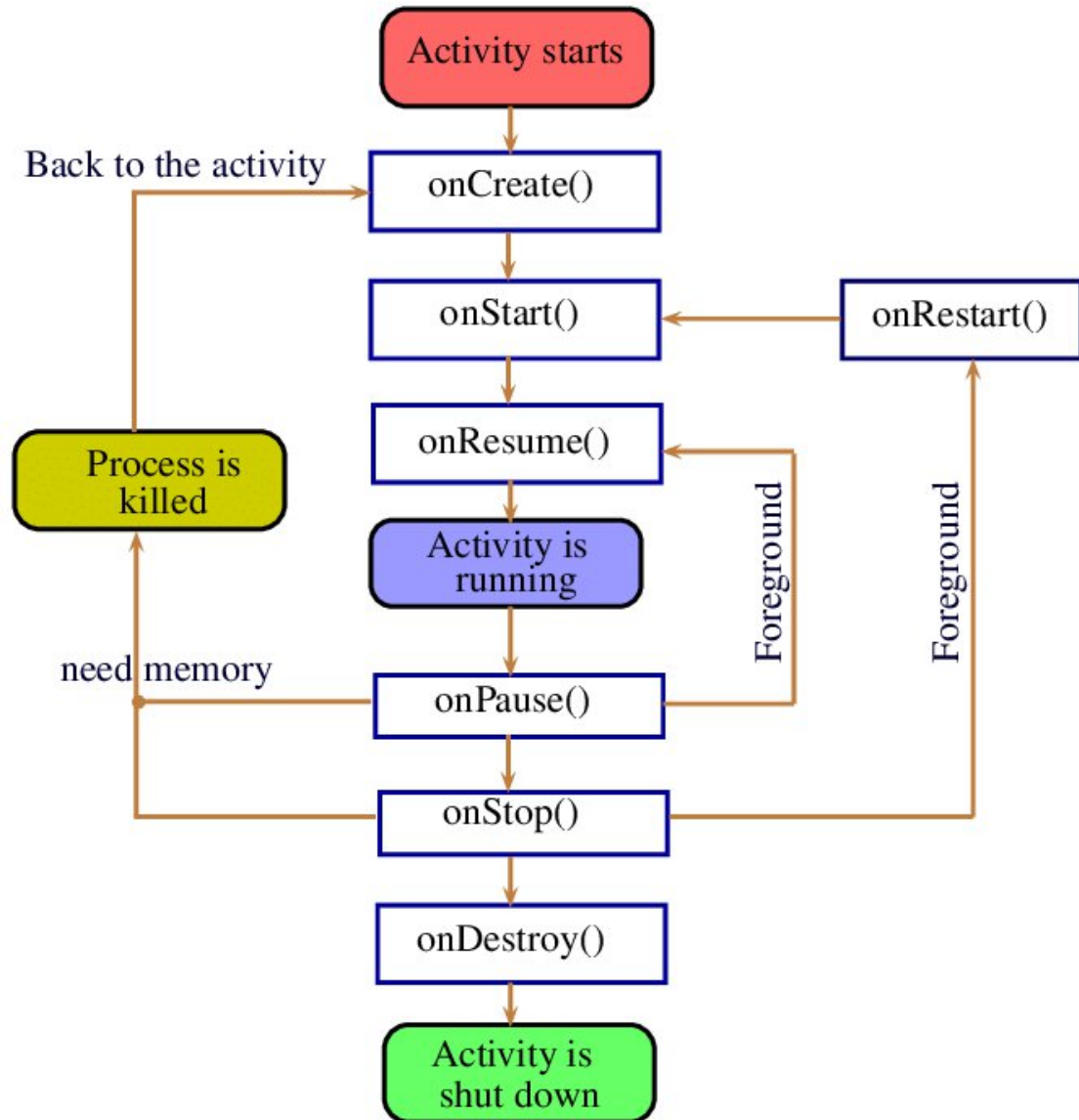
Уничтожение и сохранение состояния

Акцент на двух сценариях:

1. Уничтожение по воле пользователя (кнопка «Назад»): вызывается **onDestroy()**. Всё, активити точно умрет.

2. Уничтожение по воле системы (не хватило памяти, поворот экрана): перед **onStop()** вызывается **onSaveInstanceState()**! Сюда мы кладем данные для восстановления (в **Bundle**).

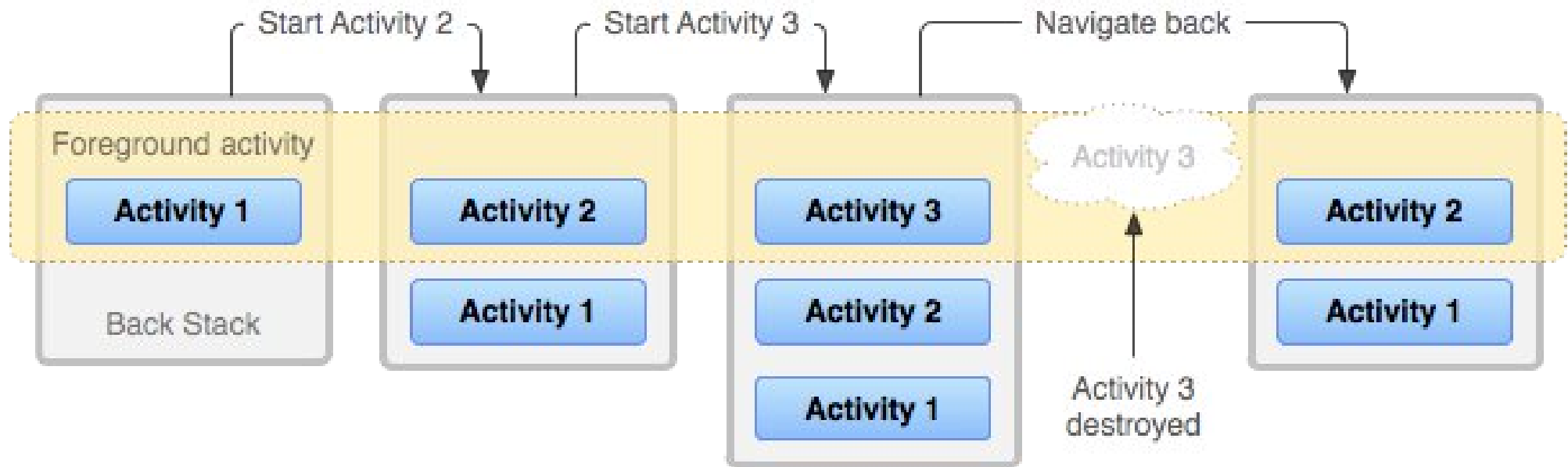
Итог: При воссоздании этот Bundle придет в **onCreate(savedInstanceState)**.



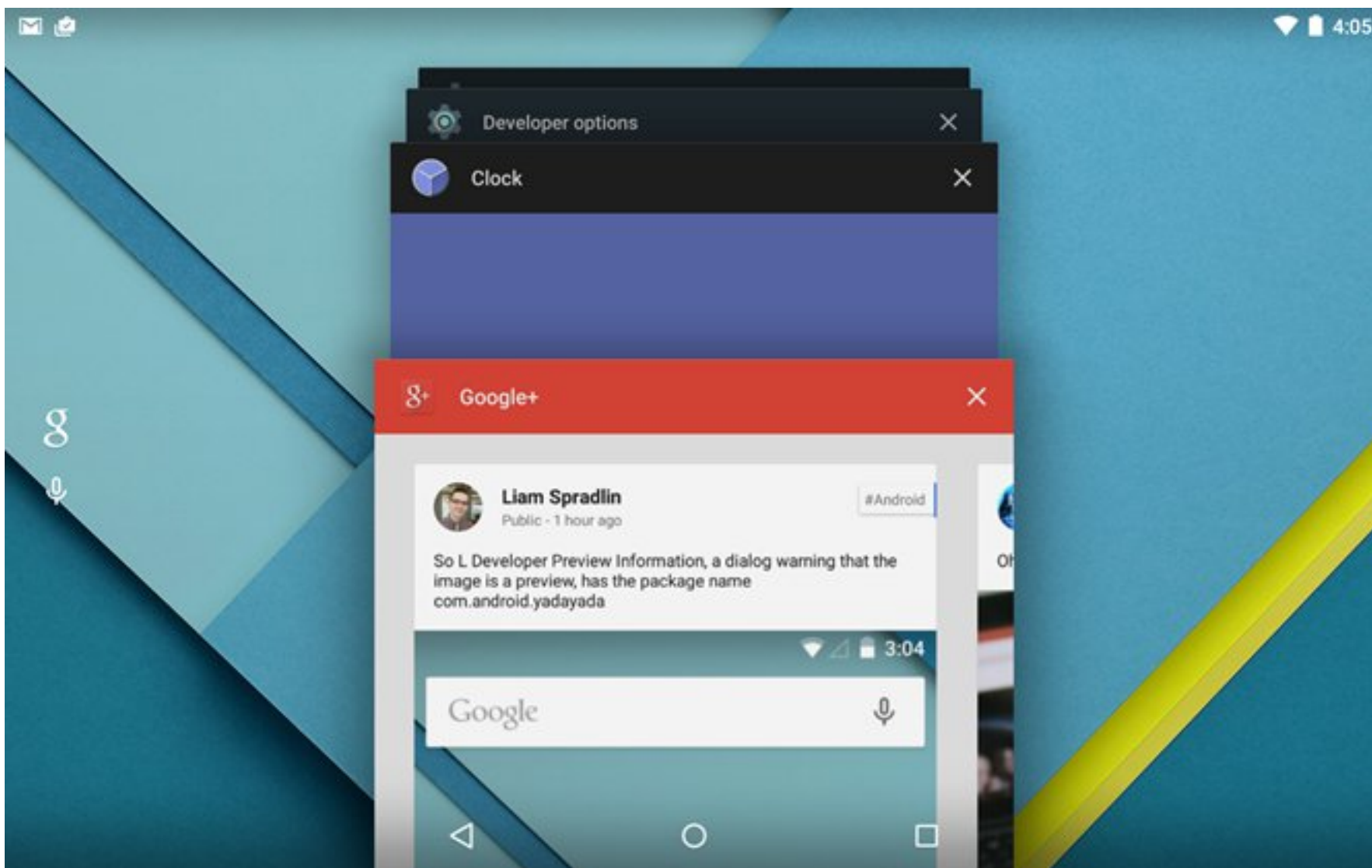
Правила хорошего тона

1. Не нагружайте onResume/onPause
2. Всегда сохраняйте UI в onSaveInstanceState
3. Освобождайте ресурсы в onPause/onStop
4. Используйте ViewModel + onSaveInstanceState

Tasks, Back Stack & Launch Modes



Навигация под капотом Android



Что делает кнопка «Назад»?



Куда?

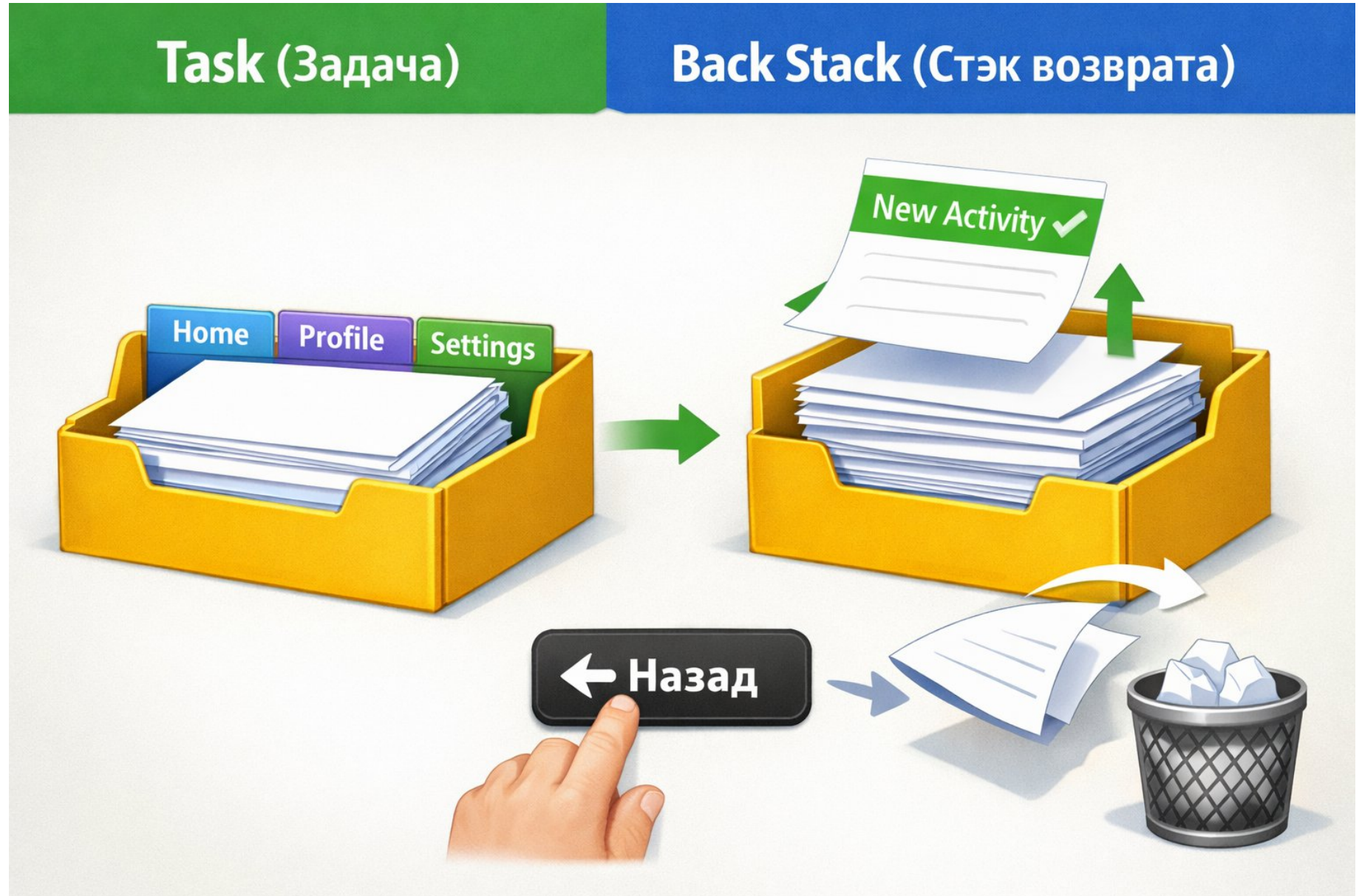
1 В предыдущее Activity

2 В главный экран

3 В другое приложение



Task (Задача) и Back Stack (Стэк возврата)



Task

Пользовательская задача

Back Stack

История её выполнения

Launch Modes (Режимы запуска)

Проблема:

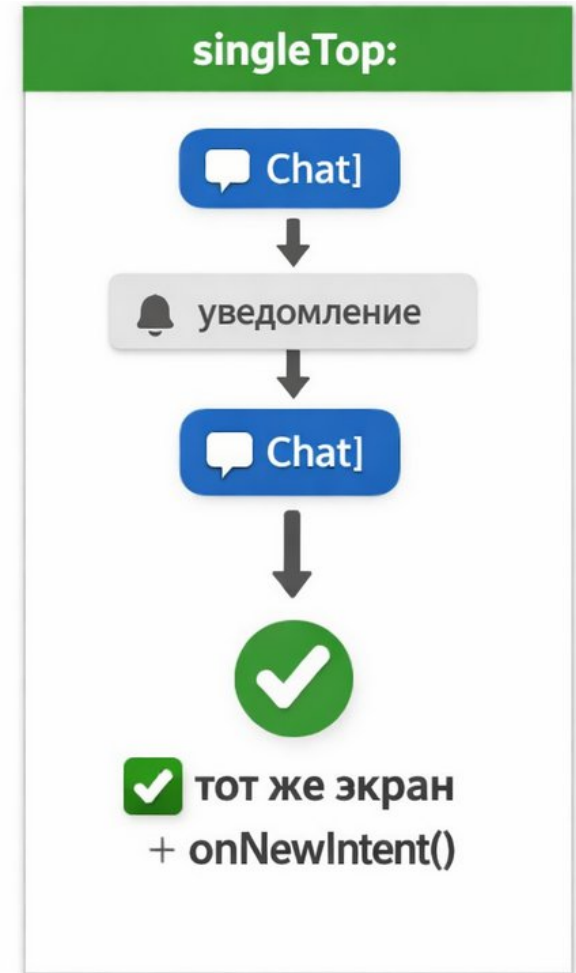
App A $\rightarrow$ App B $\rightarrow$ App A = [A1, B1, A2]

«Назад» $\rightarrow$ B $\rightarrow$ A1 $\rightarrow$ пользователь запутался!

singleTop — «Я уже наверху!»

singleTop = «Не создавай меня снова, если я уже сверху»

Пример: уведомление
→ открыть тот же чат
→ Обновляем данные,
а не создаём новый
экран!



singleTask — «Я главный!»

singleTask = «Я — главный экран. Меня может быть только один»

- **Создаёт свой Task (или использует существующий)**
- **Убирает всё, что было сверху**

Пример: Главная страница приложения

Чем отличаются **singleTop** и **singleTask**?

	singleTop	singleTask
Где проверяет?	Только верх стека	Всю задачу (Task)
Создаёт Task?	× Нет	+ Да (или использует существующий)
Удаляет экраны?	× Нет	+ Да — всё сверху удаляется
Когда использовать?	Уведомления, поиск, камера	Главная страница, логин-экран

Итог — как выбирать режим?

- + **standard** — по умолчанию (новый экран каждый раз)
- + **singleTop** — если экран может открываться на себя (уведомления, поиск)
- + **singleTask** — только для главного экрана

Services

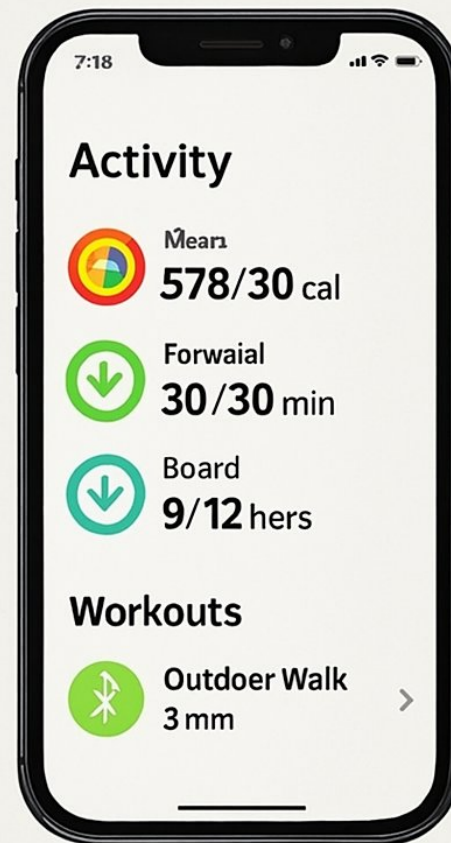
Deep dive into
Android Services



Что такое Service?

Service = Фоновая работа приложения

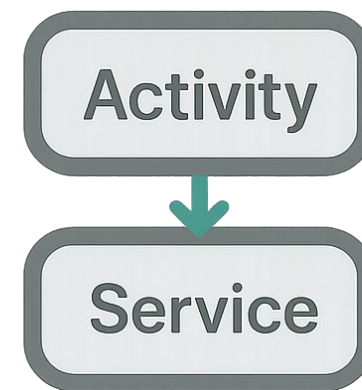
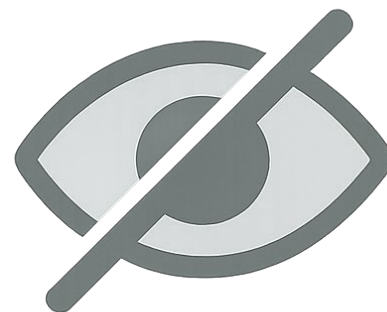
- Нет экрана (UI)
- Может работать, даже если приложение свёрнуто
- Используется для долгих задач: музыка, загрузки, синхронизация



**Activity — это лицо,
Service — невидимые руки**

Три вида Service

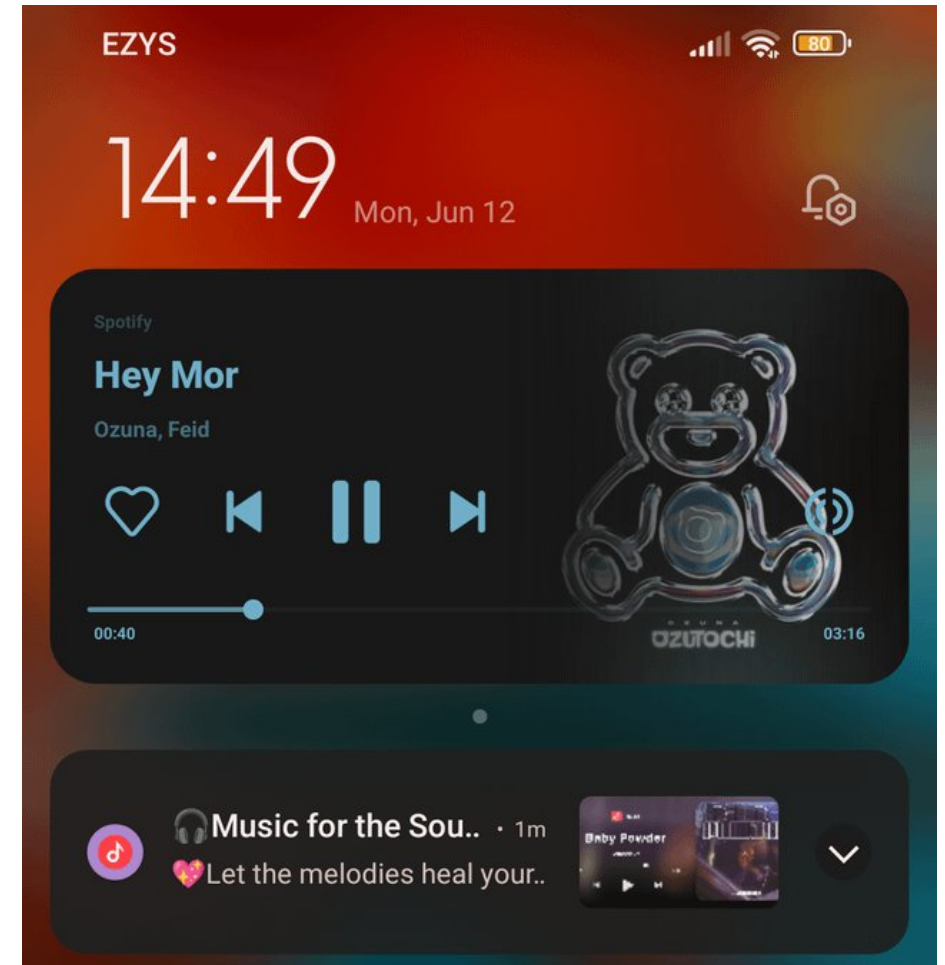
1. **Foreground Service** — с уведомлением
2. **Background Service** — без уведомления
3. **Bound Service** — привязан к компоненту



Foreground Service — когда важно не прерываться

- Высокий приоритет
- Обязательное уведомление
- Для видимой пользователю работы

Пример: музыка, навигация, запись



Как это выглядит в коде?

```
class MusicService : Service() {  
    ... override fun onStartCommand(...): Int {  
        ... val notification = createNotification()  
        ... startForeground(NOTIFICATION_ID, notification)  
        ... // Играем музыку  
        ... return START_STICKY  
    ... }  
}
```

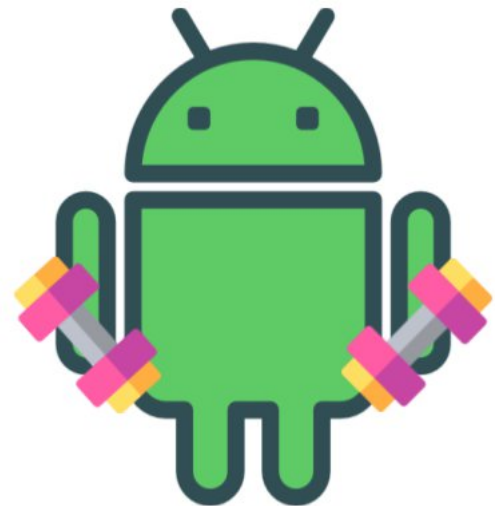
А что с фоном?

× **Background Service** — почти не работает (с Android 8+)

+ **WorkManager** — современная замена!

- Отложенные задачи
- Работает даже после перезагрузки
- Умный: ждёт Wi-Fi, зарядку и т.д.

WorkManager



WorkManager — пример кода

```
val work = OneTimeWorkRequestBuilder<UploadWorker>()  
    .setConstraints( // ← Условия  
        NetworkType.CONNECTED  
    )  
    .build()  
WorkManager.getInstance().enqueue(work)
```

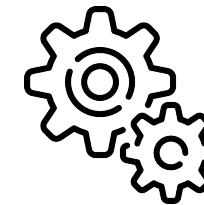
Bound Service — «живое» общение

- **Activity** подключается к **Service**
- Через него можно вызывать методы: **play()**, **pause()**
- **Service** живёт, пока есть кто-то подключён

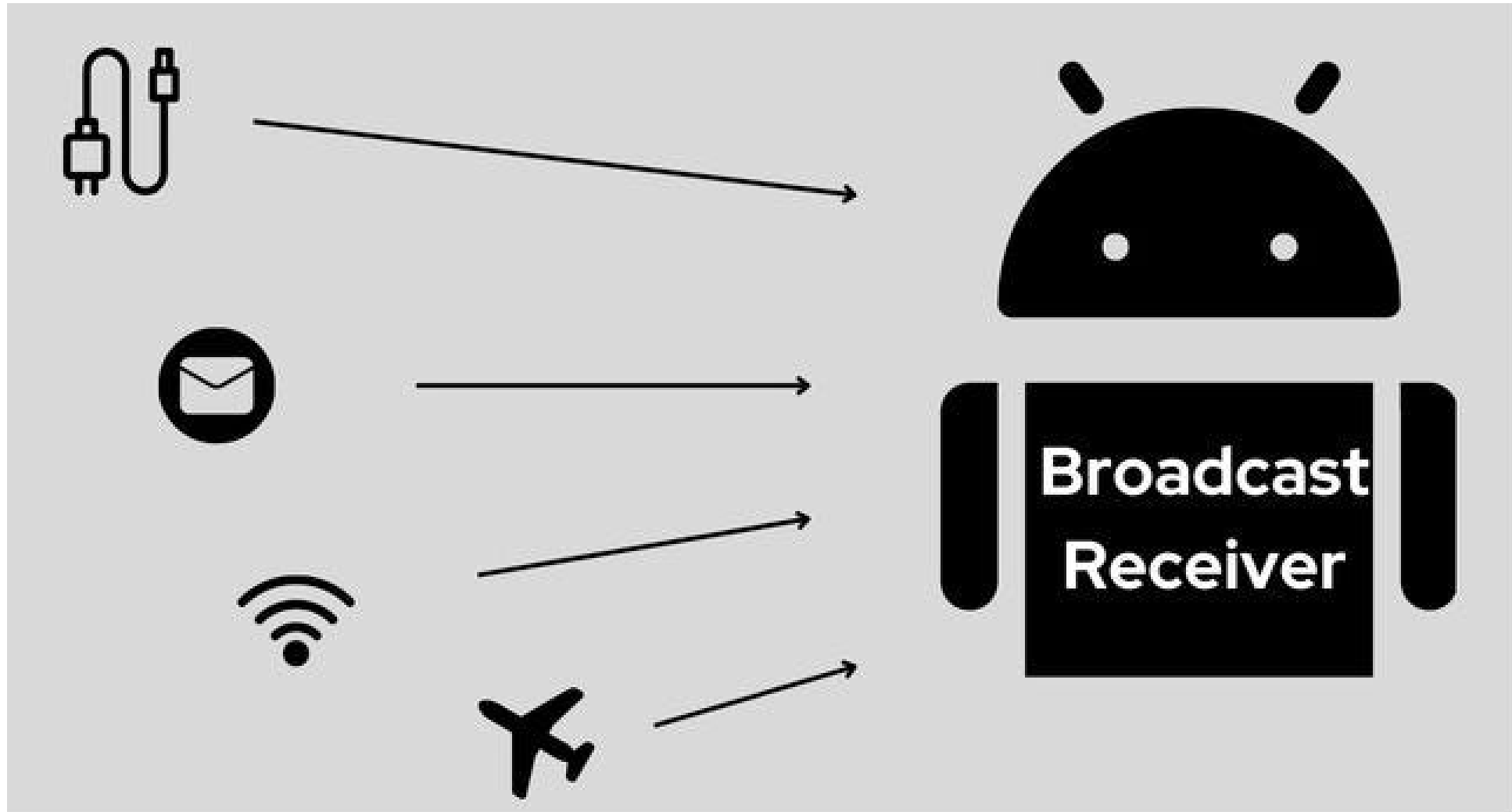
Какой Service выбрать?

 Музыка / Навигация → **Foreground Service**

 Синхронизация / Загрузка → **WorkManager**

 Управление из приложения → **Bound Service**

Broadcast Receiver



Что такое Broadcast Receiver?

Broadcast Receiver — это "уши" приложения
Он слушает системные события, например:

- Батарея почти села 
- Wi-Fi подключился 
- Пришло SMS 
- Экран выключился 



Зачем это нужно? Реальные примеры

Почему это полезно?

- Приложение останавливает загрузку, если батарея $< 10\%$
- Синхронизация данных только по Wi-Fi (чтобы не тратить мобильный интернет)
- Уведомление о важном SMS (например, от банка)
- Отключение GPS, когда экран выключен — экономия энергии



pause loading



sync



notification



GPS off

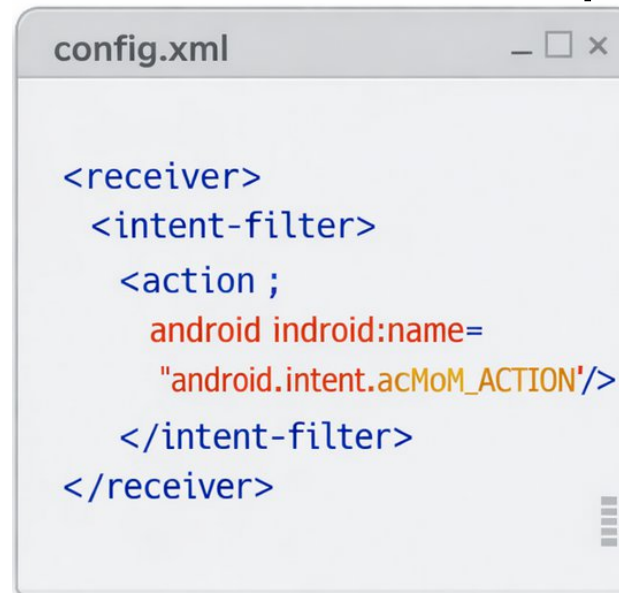
Как подключить "уши"? Два способа

Два способа подключить Receiver:

1. В файле манифеста → работает всегда

Но с Android 8+ почти нельзя использовать

2. В коде (динамически) → работает, пока открыто приложение



```
<receiver>
  <intent-filter>
    <action ;
      android:name=
        "android.intent.action.MAIN" />
    </intent-filter>
  </receiver>
```

Старый способ —
ограничен



```
class SafeReceiver :
    BroadcastReceiver() {

    override fun onReceive
        (context: Context, intent: Intent) {

        // Handle intent safely
    }
}
```

Новый способ —
безопасно

1. Создаём "уши"

```
val receiver = object : BroadcastReceiver() {  
    override fun onReceive(context: Context, intent: Intent) {  
        // Реакция: например, показать предупреждение  
    }  
}
```

**Как это
выглядит
в коде**

2. Включаем "уши" в onResume()

```
registerReceiver(receiver, IntentFilter(Intent.ACTION_BATTERY_LOW))
```

3. Выключаем в onPause()

```
unregisterReceiver(receiver)
```

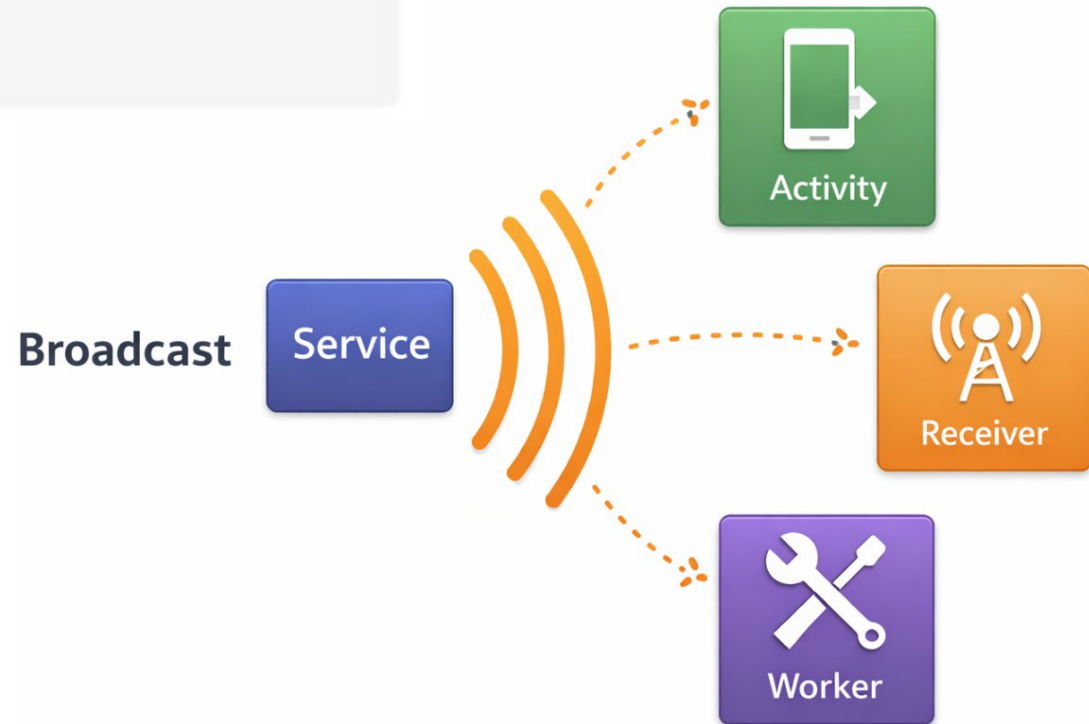


Можно слушать не только систему!

Свои собственные события

Ваше приложение тоже может **отправлять сигналы**:

```
val intent = Intent("com.myapp.UPDATE_READY")
intent.putExtra("time", System.currentTimeMillis())
sendBroadcast(intent)
```



Осторожно! Безопасность

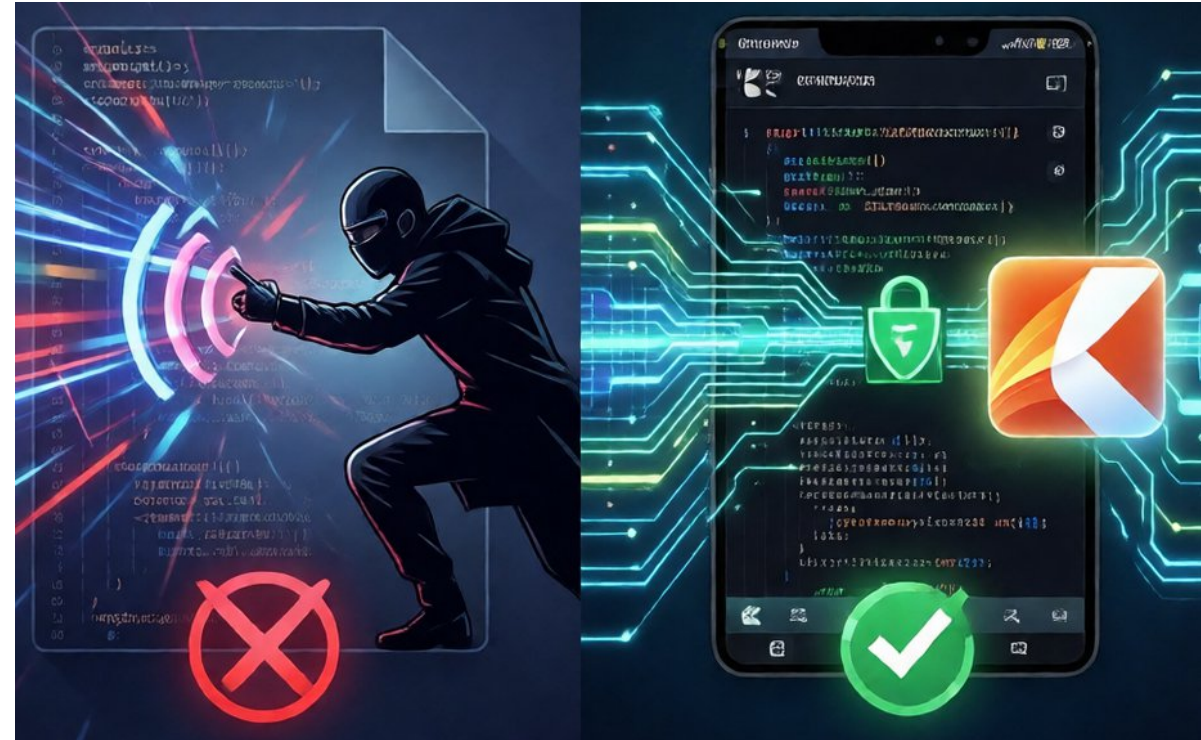
Проблема:

Любое приложение может:

- Подслушать ваш broadcast
- Подделать событие

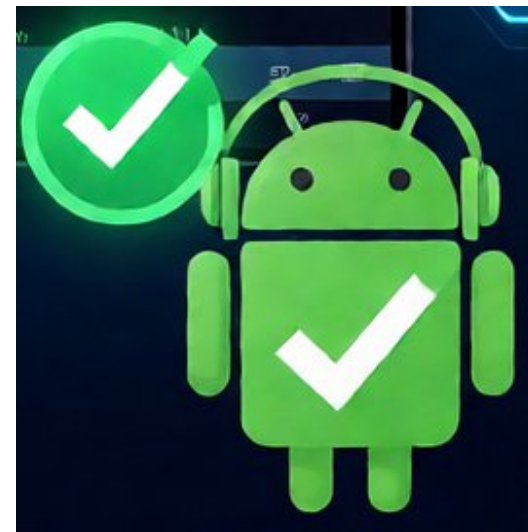
Решения:

- Не используйте broadcast для внутренних событий
- Вместо этого — LiveData или Flow (из Jetpack)
- Если всё же broadcast — защищайте permissions или подписью

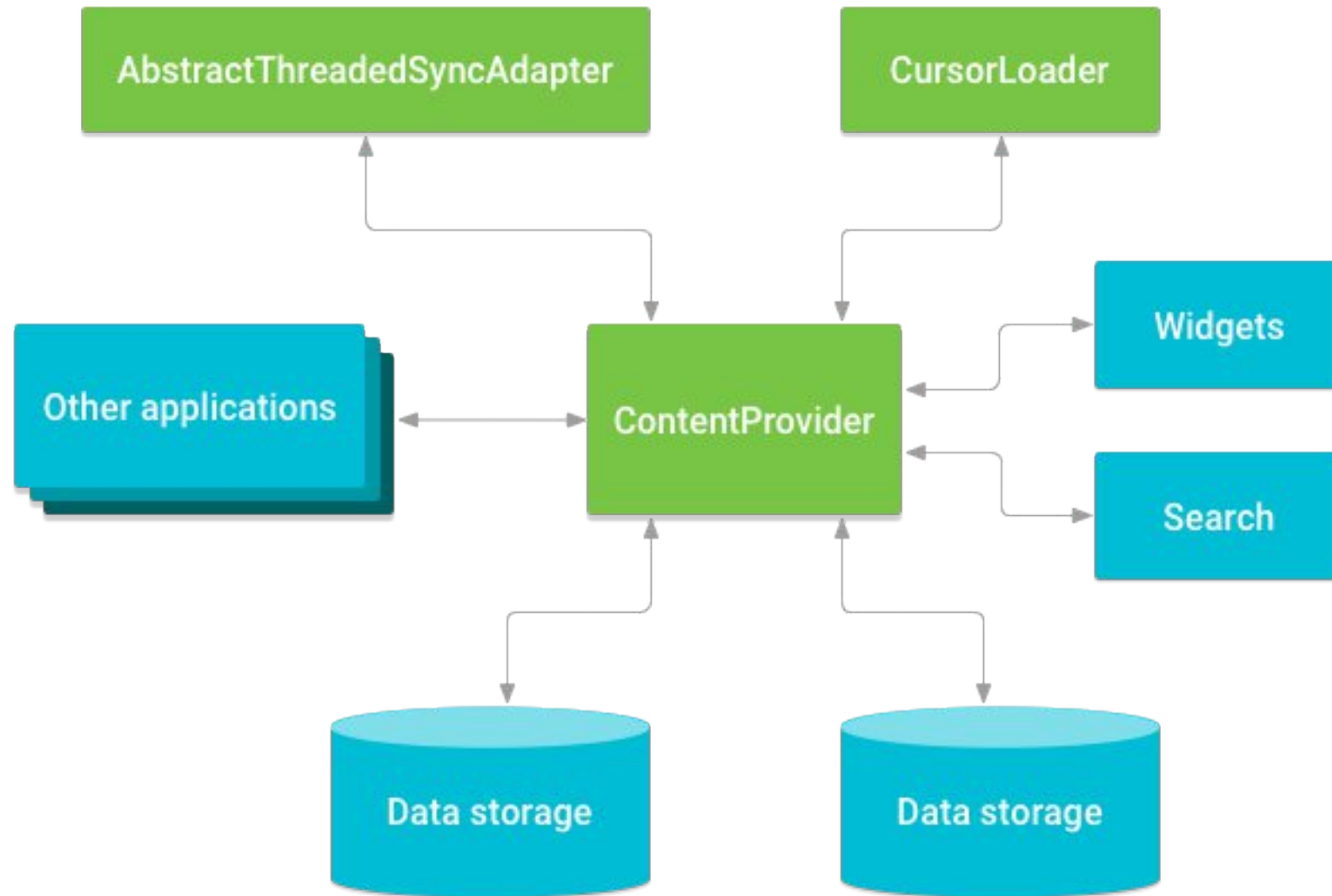


Итог по Broadcast Receivers

- Слушают системные (и свои) события
- Помогают приложению адаптироваться к изменениям
- Регистрируются в коде (а не в манифесте!)
- Не для внутреннего общения — лучше LiveData/Flow



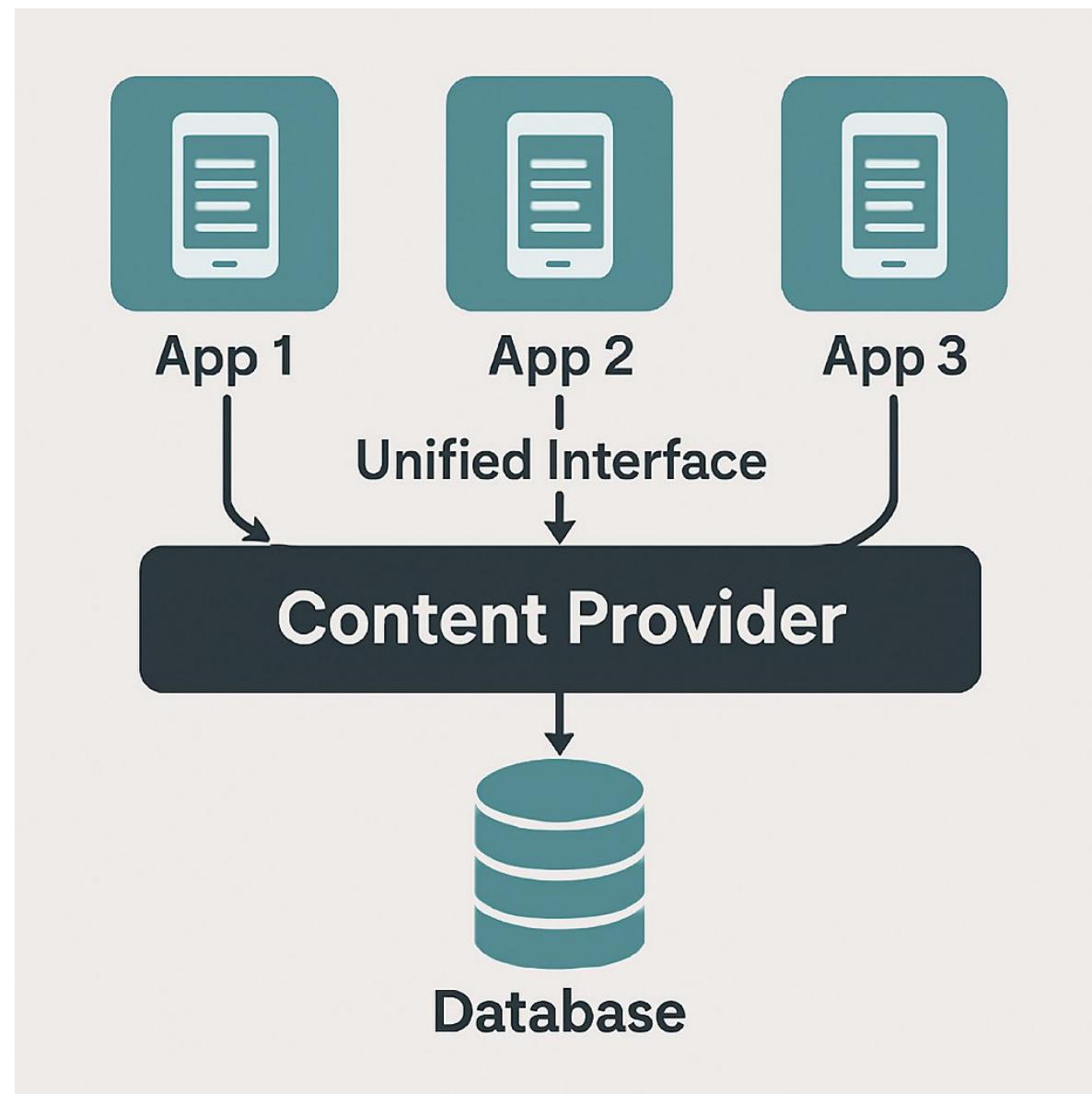
Content Provider



Что такое Content Provider?

Content Provider — это «общий интерфейс» для обмена данными между приложениями

- Позволяет читать и писать данные других приложений
- Примеры: контакты, фото, календарь
- Можно создать свой Content Provider, чтобы делиться своими данными



Как обращаться к данным? Через URI!

Content URI — это «адрес» к данным

Пример:

content://contacts/people/1

[content://]	[contacts]	[/people]	[/1]
схема	авторитет	путь	ID

- authority — кто отдаёт данные (провайдер)
- path — что именно (тип данных)
- id — конкретный контакт

Как читать данные? Например — контакты



READ_CONTACTS

Шаги для чтения данных:

1. Обратиться через ContentResolver
2. Указать URI и какие данные нужны
3. Получить Cursor — как «курсор» по списку
4. Перебрать результаты

1. Запрос к провайдеру

```
val cursor = contentResolver.query(CONTENT_URI, columns, ...)
```

2. Перебираем результаты

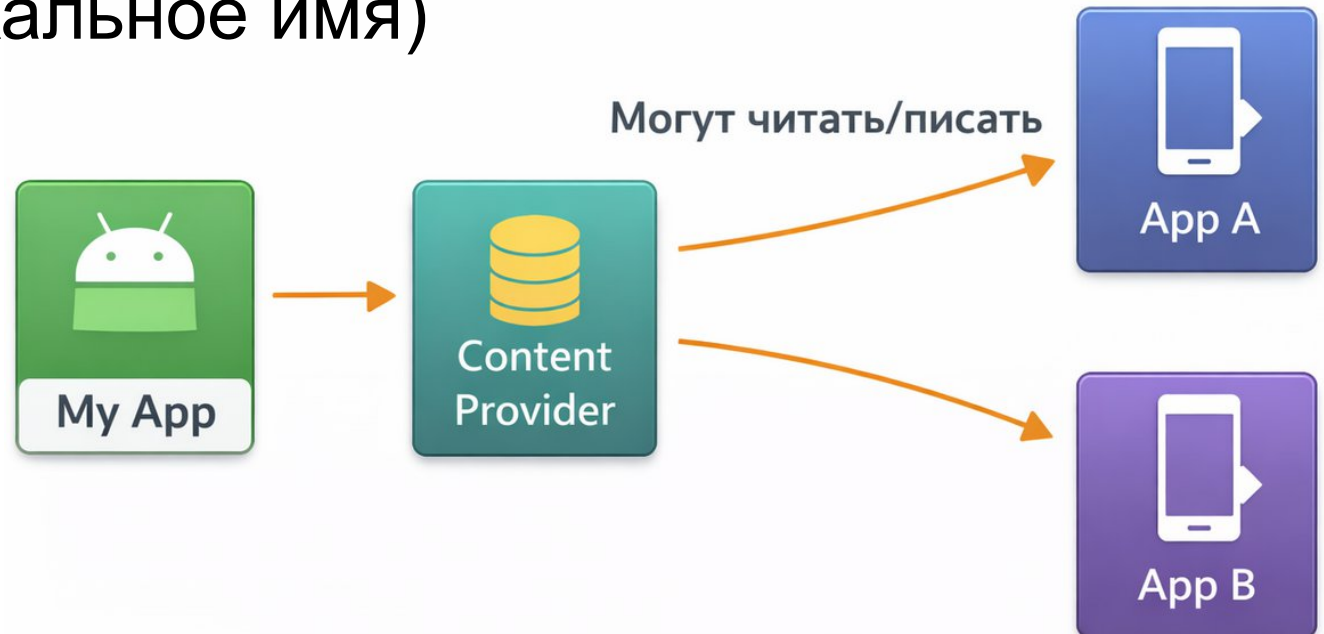
```
while (cursor.moveToNext()) {  
    val name = cursor.getString(0)  
    println(name)  
}
```

А можно сделать свой Content Provider?

Да! Если хотите делиться данными с другими приложениями

Что нужно сделать:

- Создать класс, унаследованный от `ContentProvider`
- Реализовать 4 метода: `query`, `insert`, `update`, `delete`
- Зарегистрировать в `AndroidManifest.xml`
- Придумать свой `authority` (уникальное имя)



Безопасность! Кто может читать ваши данные?

Контролируйте доступ через разрешения

- В манифесте можно указать:
- **android:readPermission** — кто может читать
- **android:writePermission** — кто может писать

Пример:

```
<provider
    . . . android:name=".MyProvider"
    . . . android:authorities="com.myapp.provider"
    . . . android:readPermission="com.myapp.READ"
    . . . android:writePermission="com.myapp.WRITE" />
```

Когда использовать? А когда — нет?

Используйте Content Provider если:

- ✓ Другие приложения **должны** получить доступ к вашим данным
- ✓ Нужна интеграция с системой (поиск, виджеты, голосовые команды)

НЕ используйте для:

- Данные нужны **только внутри вашего приложения**
- Просто храните данные — тогда лучше **Room** или **SharedPreferences**

Что такое Room?

Room — это:

- Библиотека от Google
 - Удобная работа с базой данных
- Только внутри одного приложения**



Пример Room

Пример использования Room:

- Список задач
- Заметки
- Избранные товары



Главное отличие

Room	Content Provider
Для одного приложения	Для нескольких приложений
Удобство	Стандартизация
Простота	Безопасный доступ

Можно ли использовать вместе?

Да, можно:

- Room — внутри
- Content Provider — наружу

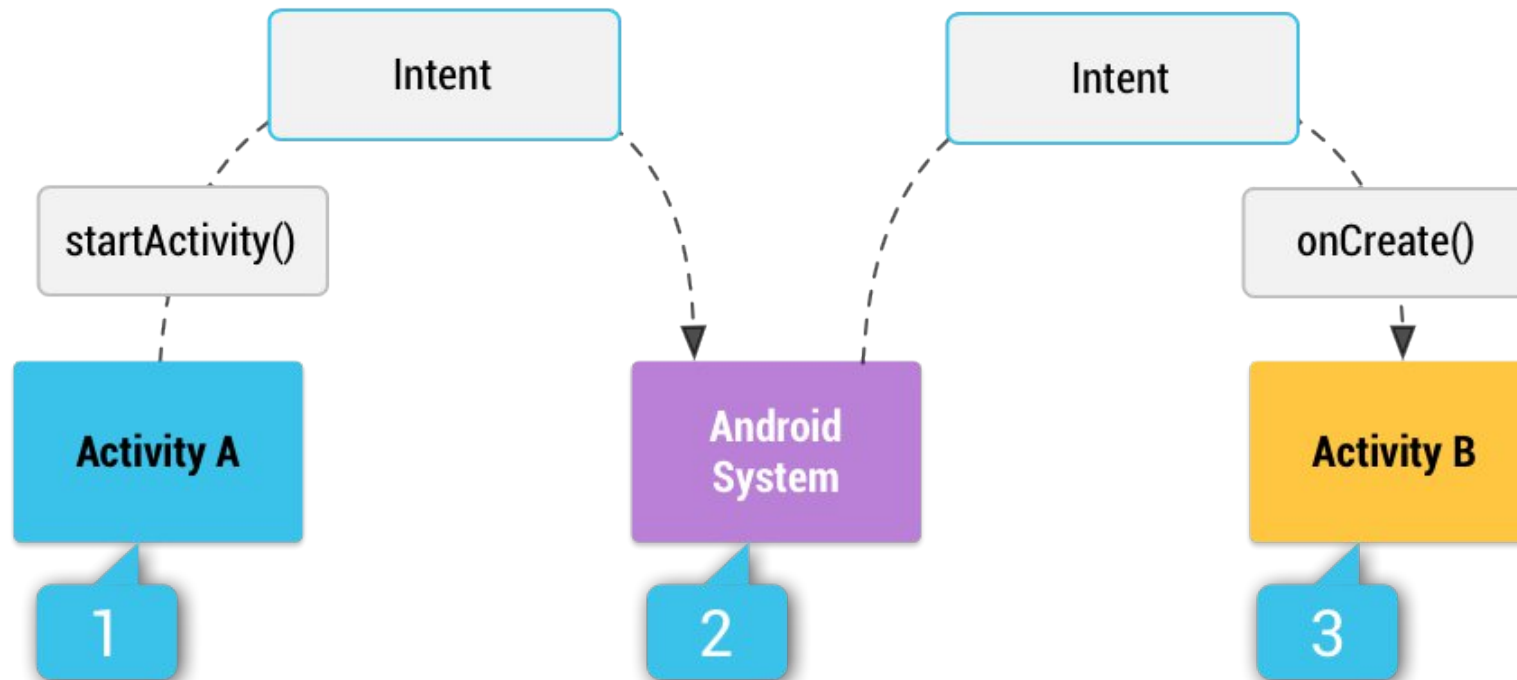


Взаимодействие компонентов

Intent — связь между компонентами

Intent = «намерение»

- **Explicit Intent** — «Запусти именно этот экран»
- **Implicit Intent** — «Кто-то открой это!»



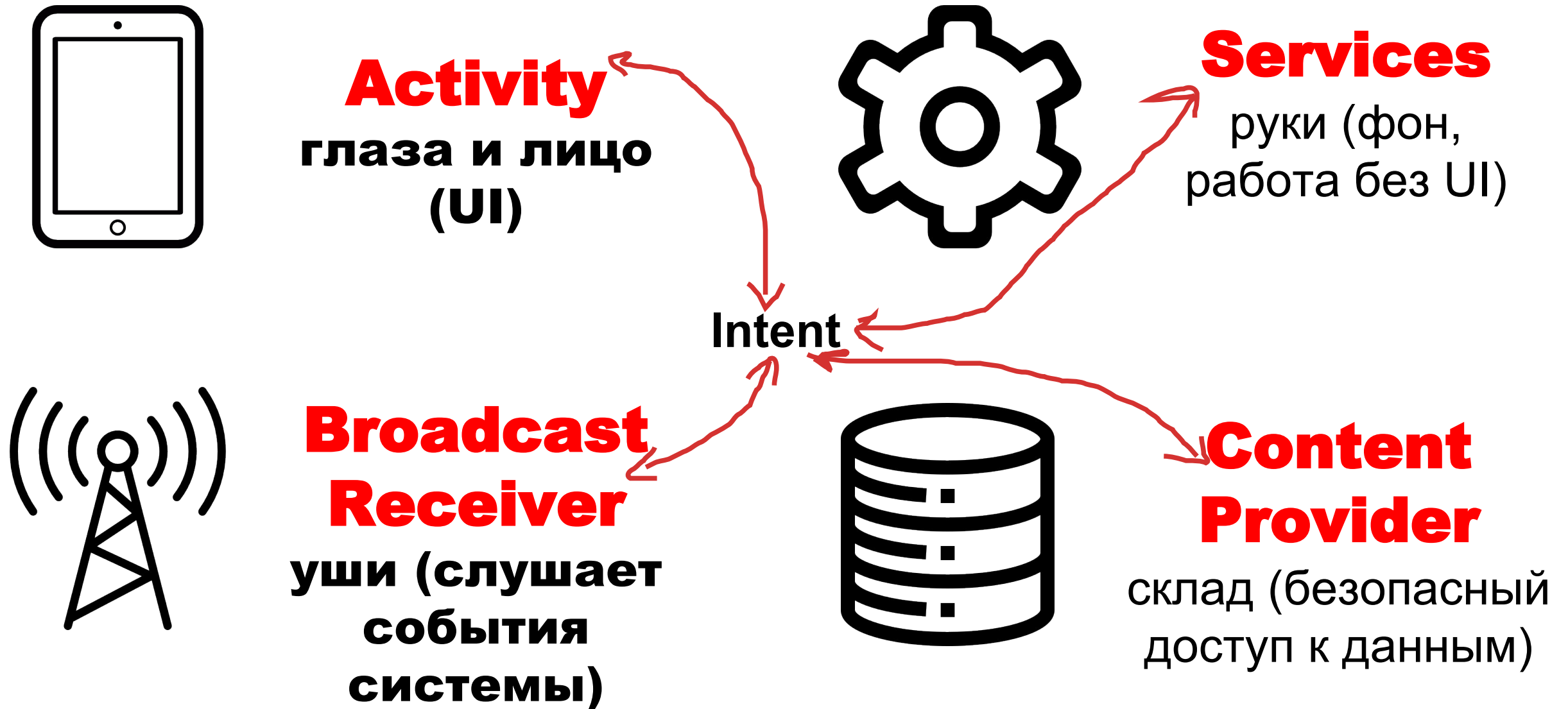
Как приложение «заявляет» о своих способностях? → Intent Filters

Объявляем: 'Я умею это делать!'

В AndroidManifest.xml:

```
<intent-filter>
    <action android:name="android.intent.action.VIEW" />
    <category android:name="android.intent.category.DEFAULT" />
    <data android:scheme="https" />
</intent-filter>
```

Четыре компонента Android



"Социальная сеть"

Архитектура соцсети на Android

Компонент	Функция в соцсети
Activity	Экран ленты, профиля, загрузки фото
Service	Фоновая загрузка фото / синхронизация
BroadcastReceiver	Получение push-уведомлений о лайках / комментариях
ContentProvider	Доступ к контактам из телефонной книги для приглашения друзей